

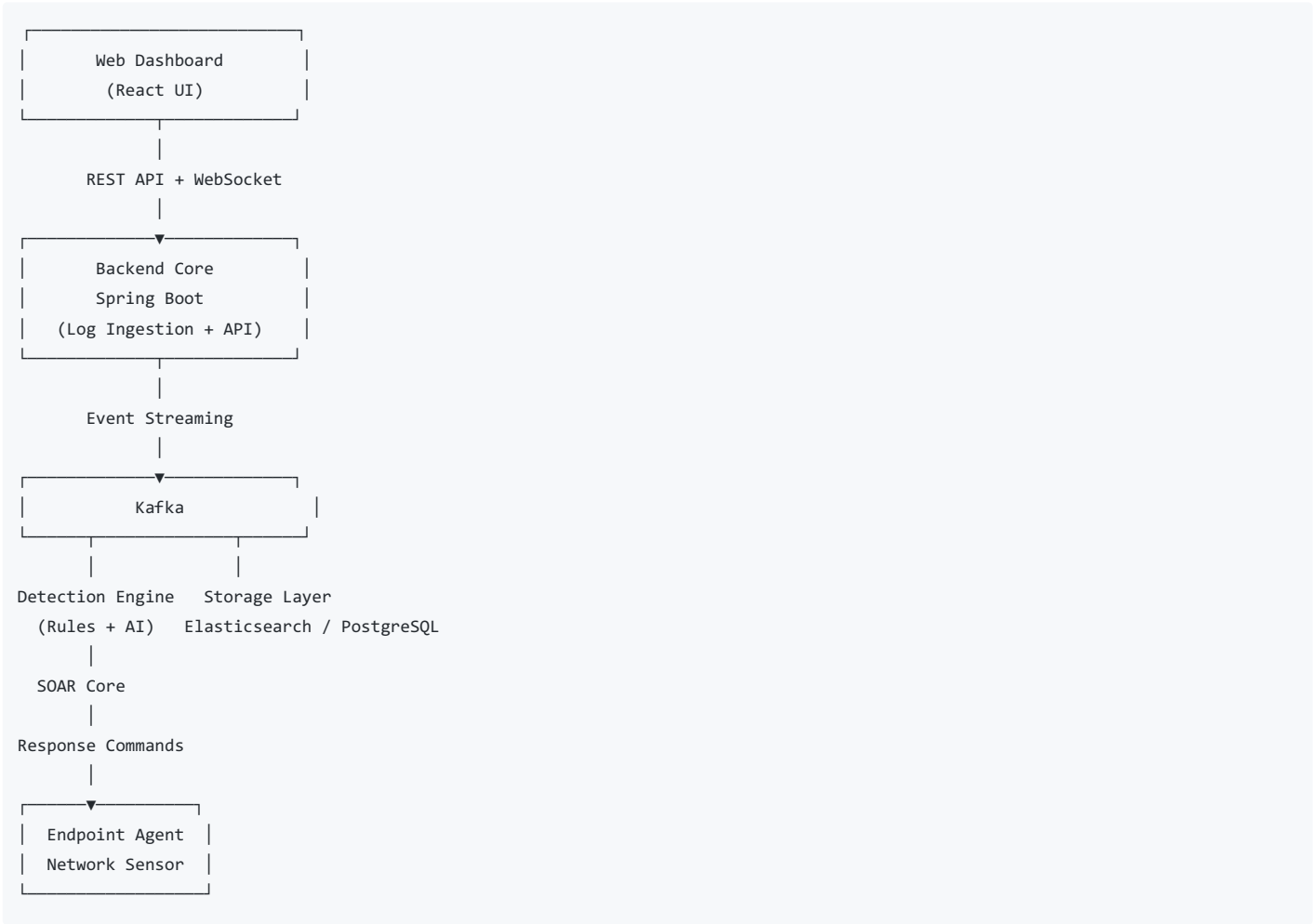
🔒 NetSpect — Unified Autonomous SOC Platform

A full-stack Security Operations Center (SOC) platform built for SMEs, providing automated threat detection and response.

📋 Project Overview

NetSpect is an autonomous SOC platform that combines NDR, EDR, SIEM, SOAR, and AI into one unified system. It monitors network traffic and endpoint devices, detects attacks automatically, and responds to them in real time without human intervention.

🏗️ System Architecture



📦 System Modules

Module	Role
NDR	Network traffic analysis
EDR	Endpoint device monitoring
SIEM	Log collection and analysis
SOAR	Automated response execution
AI Engine	Detects abnormal behaviors

📁 Backend — Completed Sprints

Sprint 1 — Event Ingestion API

The entry gate of the system. Receives all events from NDR and EDR, normalizes them into a unified schema, and pushes them into Kafka.

Endpoints:

```
POST /api/events/network    → NDR / Suricata events
POST /api/events/endpoint   → EDR / Agent events
POST /api/events/login      → Login events
```

Example Payload:

```
{
  "deviceId": "server1",
  "eventType": "process_start",
  "process": "powershell.exe",
  "user": "admin"
}
```

Kafka Topics Created:

```
events.network
events.endpoint
events.alerts
```

Sprint 2 — Storage & Query Layer

Consumes events from Kafka and stores them in both Elasticsearch and PostgreSQL. Provides a Query API for the Dashboard.

Query Endpoints:

```
GET /api/query/events          → all events
GET /api/query/events/device/{deviceId} → by device
GET /api/query/events/severity/{severity} → by severity
GET /api/query/events/source/{source} → NDR or EDR
GET /api/query/events/type/{eventType} → by event type
GET /api/query/stats           → summary stats
```

Stats Response Example:

```
{
  "totalEvents": 3,
  "highSeverity": 1,
  "mediumSeverity": 1,
  "lowSeverity": 1,
  "ndrEvents": 1,
  "edrEvents": 2
}
```

Sprint 3 — SOAR + WebSocket Live Alerts

Receives triggers from the Detection Engine and sends automated response commands to Endpoint Agents. Pushes live alerts to the Dashboard in real time via WebSocket.

SOAR Endpoints:

```
POST /api/soar/kill-process → kill suspicious process
POST /api/soar/block-ip    → block dangerous IP
POST /api/soar/isolate     → isolate compromised machine
GET  /api/soar/commands   → all executed commands
GET  /api/soar/stats       → commands summary
POST /api/soar/test-alert  → push test alert to Dashboard
```

WebSocket:

```
Connect to:    /ws/alerts
Subscribe to:  /topic/alerts
Test page at:  http://localhost:8080/test.html
```

SOAR Stats Example:

```
{
  "totalCommands": 3,
  "killProcessCount": 1,
  "blockIpCount": 1,
  "isolateHostCount": 1
}
```

📁 Project Structure

```

NetSpect/
├── docker-compose.yml
├── pom.xml
├── mvnw / mvnw.cmd
├──
└── src/
    ├── main/
    │   ├── java/com/soc/backend_core/
    │   │   ├── BackendCoreApplication.java      ← App entry point
    │   │   ├── KafkaConfig.java                ← Kafka topics config
    │   │   ├── WebSocketConfig.java            ← WebSocket/STOMP config
    │   │   ├──
    │   │   ├── controller/
    │   │   │   ├── EventController.java        ← Ingestion endpoints
    │   │   │   ├── EventQueryController.java   ← Query endpoints
    │   │   │   ├── SoarController.java         ← SOAR response endpoints
    │   │   │   └── WebSocketController.java     ← WebSocket test endpoint
    │   │   ├──
    │   │   ├── dto/
    │   │   │   ├── NetworkEventRequest.java    ← NDR event input
    │   │   │   ├── EndpointEventRequest.java   ← EDR event input
    │   │   │   ├── LoginEventRequest.java      ← Login event input
    │   │   │   └── SoarCommandRequest.java     ← SOAR command input
    │   │   ├──
    │   │   ├── Entities/
    │   │   │   ├── domain/
    │   │   │   │   ├── UnifiedEvent.java       ← Normalized event model
    │   │   │   │   ├── AlertMessage.java       ← WebSocket alert model
    │   │   │   │   └── SoarCommand.java         ← SOAR command model
    │   │   │   ├── elastic/
    │   │   │   │   └── EventDocument.java       ← Elasticsearch document
    │   │   │   └── jpa/
    │   │   │       └── EventRecord.java         ← PostgreSQL entity
    │   │   ├──
    │   │   ├── producer/
    │   │   │   └── EventProducer.java           ← Publishes to Kafka
    │   │   ├──
    │   │   ├── consumer/
    │   │   │   └── EventConsumer.java           ← Consumes from Kafka
    │   │   ├──
    │   │   ├── service/
    │   │   │   ├── EventStorageService.java     ← Saves to ES + PostgreSQL
    │   │   │   ├── EventQueryService.java       ← Queries ES + PostgreSQL
    │   │   │   └── SoarService.java             ← SOAR logic
    │   │   ├──
    │   │   └── repository/
    │   │       ├── elastic/
    │   │       │   └── EventDocumentRepository.java ← Elasticsearch repo
    │   │       ├── jpa/
    │   │       │   └── EventRecordRepository.java  ← PostgreSQL repo
    
```

🔧 Tech Stack

Technology	Version	Purpose
Spring Boot	3.5.1	REST API framework
Apache Kafka	7.5.0	Event streaming
Elasticsearch	8.12.0	Fast event search and

Technology	Version	storage Purpose
PostgreSQL	15	Structured event storage
WebSocket (STOMP)	-	Live alerts to Dashboard
Docker	Latest	Running all services
Java	21	Programming language

🔧 How to Run

Step 1 — Start All Services with Docker

```
cd NetSpect
docker-compose up -d
```

This starts:

- Kafka + Zookeeper
- Elasticsearch (port 9200)
- PostgreSQL (port 5432)

Step 2 — Start the Backend

```
./mvnw spring-boot:run
```

Step 3 — Verify Everything is Running

```
docker ps
```

You should see:

```
kafka
zookeeper
elasticsearch
postgres
```

Spring Boot runs on:

```
http://localhost:8080
```

🧪 Quick Test

Send a Network Event

```
curl -X POST http://localhost:8080/api/events/network \
-H "Content-Type: application/json" \
-d '{
  "deviceId": "sensor1",
  "eventType": "port_scan",
  "srcIp": "192.168.1.99",
  "destIp": "192.168.1.10"
}'
```

Send an Endpoint Event

```
curl -X POST http://localhost:8080/api/events/endpoint \
-H "Content-Type: application/json" \
-d '{
  "deviceId": "windows-pc1",
  "eventType": "malware_detected",
  "process": "malware.exe",
  "user": "admin"
}'
```

Trigger SOAR Response

```
curl -X POST http://localhost:8080/api/soar/kill-process \
-H "Content-Type: application/json" \
-d '{
  "deviceId": "windows-pc1",
  "targetProcess": "malware.exe",
  "triggeredBy": "malware_detection"
}'
```

View Live Alerts in Browser

<http://localhost:8080/test.html>

🔍 Final Demo Scenario

1. Attacker scans network
2. NDR detects the scan → POST /api/events/network
3. Attacker drops malware
4. EDR detects the malware → POST /api/events/endpoint
5. SIEM correlates both events → Kafka + Elasticsearch
6. SOAR responds automatically → POST /api/soar/kill-process
→ POST /api/soar/block-ip
→ POST /api/soar/isolate
7. Dashboard shows live alerts → WebSocket /ws/alerts